

Architectural Blueprint for a Universal Voxel Serialization to JSON Specification Converter

The serialization of three-dimensional voxel data has undergone a profound evolution over the last decade, driven by the increasing complexity of rendering engines, the introduction of infinite block states, and the constant demand for highly efficient volumetric storage. To engineer a universal converter capable of ingesting any legacy or modern schematic file format and outputting a standardized, human-readable JSON specification—referred to herein as the jsonspec schema—a deep morphological and mathematical analysis of existing binary formats is required. This document provides an exhaustive architectural and algorithmic blueprint for translating highly compressed, bit-packed, and multi-layered voxel arrays into a flattened, coordinate-mapped JSON object array.

The analysis examines the core Named Binary Tag (NBT) encoding layer, parses the structural paradigms of the MCEdit, Sponge, Litematica, Axiom, and Bedrock formats, and defines the mathematical models necessary for coordinate resolution, data normalization, and memory-efficient streaming serialization.

Foundations of Named Binary Tag (NBT) Encoding and Compression

The vast majority of voxel-based schematic formats rely on the Named Binary Tag (NBT) specification, a format originally engineered to store hierarchical game data in a highly compact, structured binary form. Before any block data can be mapped to an intermediary JSON structure, the ingestion pipeline of the universal converter must successfully navigate the complexities of NBT encoding, compression wrappers, and platform-specific byte ordering. NBT data is constructed as a tree of tags. Each tag is prefixed by a single-byte type identifier, followed by a two-byte length-prefixed Modified UTF-8 (MUTF-8) string denoting the tag's name. The payload of the tag follows immediately after the name. The exact binary representation of this data fluctuates wildly depending on the game edition and the specific community tool utilized to generate the schematic.

The core NBT tag structures are defined by their numerical identifiers, which the converter's parsing engine must rigorously evaluate to traverse the binary tree without encountering offset misalignments.

Tag ID	Name	Payload Description	Storage Capacity
0	TAG_End	None. Used to signify the termination of a TAG_Compound. It possesses no name.	N/A
1	TAG_Byte	A single signed 8-bit integer.	-128 to 127
2	TAG_Short	A signed 16-bit integer.	-32,768 to 32,767

Tag ID	Name	Payload Description	Storage Capacity
3	TAG_Int	A signed 32-bit integer.	-2,147,483,648 to 2,147,483,647
4	TAG_Long	A signed 64-bit integer.	-9.22E18 to 9.22E18
5	TAG_Float	A 32-bit single-precision floating-point number (IEEE 754).	Standard IEEE 754
6	TAG_Double	A 64-bit double-precision floating-point number (IEEE 754).	Standard IEEE 754
7	TAG_Byte_Array	A 32-bit integer defining length, followed by an array of 8-bit bytes.	Varies by JVM limits
8	TAG_String	A 16-bit integer defining length, followed by UTF-8 string data.	Up to 32,767 bytes
9	TAG_List	A 1-byte tag ID defining the list type, a 32-bit length integer, and the sequential payloads.	Varies by JVM limits
10	TAG_Compound	A recursive dictionary of fully formed tags, terminated by a TAG_End.	Virtual memory limits
11	TAG_Int_Array	A 32-bit integer defining length, followed by an array of 32-bit integers.	Varies by JVM limits
12	TAG_Long_Array	A 32-bit integer defining length, followed by an array of 64-bit integers.	Varies by JVM limits

Java Edition formats, encompassing MCEdit, Sponge, and Litematica, strictly adhere to big-endian byte ordering. Furthermore, these files are almost exclusively compressed. While the original NBT specification dictates the use of the Gzip algorithm, certain modern implementations and community tools utilize Zlib compression or leave the data uncompressed entirely to reduce CPU overhead during parsing.

A robust converter must dynamically determine the compression wrapper by inspecting the file's initial magic bytes prior to engaging the NBT parser. A binary file beginning with the hexadecimal sequence 1F 8B indicates Gzip compression, which utilizes a CRC-32 checksum and an 18-byte wrapper. Conversely, a sequence beginning with 78 indicates Zlib compression, utilizing the faster Adler-32 checksum and a minimal 6-byte wrapper. Any other byte sequence suggests an uncompressed payload, requiring direct binary stream analysis. Standardizing the decompression phase requires careful deployment of decompression algorithms; for example, the zlib and gzip modules in higher-level languages must be instantiated with specific window bit parameters (wbits) to correctly interpret headers without throwing "incorrect header check" errors.

Conversely, Bedrock Edition schematics, commonly saved with the `.mcstructure` extension, operate on a fundamentally different paradigm. These files utilize little-endian byte ordering and are typically stored entirely uncompressed, resulting in significantly larger file sizes on disk relative to their geometric volume. Furthermore, network transmissions in Bedrock occasionally utilize a "Network Little Endian" format that encodes 32-bit and 64-bit integers using variable-length integer (varint) encoding to artificially restrict payload size during packet serialization.

The universal converter must therefore implement an adaptable NBT parser capable of swapping endianness and dynamically decoding varints based on the detected file signature and target platform. Utilizing established abstraction layers, such as `nbtlib`, allows for seamless transition between big-endian and little-endian interpretation simply by setting the `byteorder="little"` parameter during stream initialization.

Morphological Analysis of Source Schematic Formats

The core challenge of the universal converter lies in translating the diverse storage paradigms utilized across different eras of voxel data serialization. The target `jsonspec` schema requires an explicit, one-dimensional array of coordinate objects, each containing an `x`, `y`, `z` coordinate and a string representation of the block palette index. To achieve this output consistently, the ingestion pipeline must support and normalize five primary architectural paradigms.

The Legacy Paradigm: MCEdit and Schematica

Prior to the massive architectural overhaul known as the "Flattening" in Java Edition 1.13, blocks were universally identified by a primary 8-bit numerical ID and a supplementary 4-bit metadata value. This structural limitation restricted the game to 256 base block types and 16 variants per block. The legacy MCEdit `.schematic` format, alongside older versions of the Schematica mod, strictly reflects this era.

The legacy format does not utilize a localized block palette or string-based namespaces. Instead, it stores voxel data in two parallel, uncompressed byte arrays nested within the NBT compound: the `Blocks` array and the `Data` array. The spatial dimensions of the schematic are defined by standard `Width`, `Height`, and `Length` short integer tags. The total geometric volume directly dictates the exact length of both byte arrays.

Because the legacy format relies on hardcoded numerical identifiers that no longer exist in modern implementations, the converter architecture requires a robust, static translation matrix. This matrix must map legacy ID and metadata pairs—such as ID 17, Data 1 representing a Spruce Log—into modern, string-based Resource Locations like `minecraft:spruce_log`. This translation layer is critical for ensuring that the resulting JSON palette conforms to modern naming conventions. Advanced implementations should leverage mechanisms analogous to Mojang's DataFixerUpper (DFU) to safely migrate and upgrade these legacy palettes into the modern schema without losing intricate block state data.

The Varint-Packed Architecture: Sponge Schematic Specification

The Sponge Schematic format was engineered specifically to solve the forward-compatibility issues inherent to the MCEdit format. By introducing a localized `Palette` object and entirely eliminating hardcoded numerical IDs, the Sponge format allowed for infinite mod compatibility

and dynamic block states. The current specifications, versions 2 and 3, map full block state strings (including properties like facing=north and half=top) to localized integer indices. The primary storage mechanism in the Sponge format is the BlockData array, which stores the localized palette indices for every voxel within the defined volume. To achieve maximum compression, this array is encoded as a varint[] (variable-length integer array) wrapped inside a standard NBT Byte Array.

The varint decoding algorithm is a critical component of the converter's parsing engine. It requires iterating through the byte array and processing the lower 7 bits of each byte. If the most significant bit (MSB) of the byte is set to 1, the algorithm continues to the next byte, accumulating the bits using bitwise left-shift operations multiplied by 7. Once a byte is encountered with an MSB of 0, the complete integer has been extracted, representing the localized palette index for a specific spatial coordinate.

The spatial mapping of the Sponge format utilizes a highly specific linear indexing formula. The one-dimensional index i corresponds to the three-dimensional coordinates (x , y , z) based on the following equation:

$$i = x + (z \times \text{Width}) + (y \times \text{Width} \times \text{Length})$$

This mathematical relationship indicates that the X axis varies the fastest in memory, followed by the Z axis, and finally the Y axis. The universal converter must reverse this calculation sequentially to populate the jsonspec coordinate objects accurately.

Sponge version 3 introduces further complexities by isolating biome data. While version 2 utilized a two-dimensional biome array, version 3 supports complete three-dimensional biome palettes, allowing the converter to extract sub-chunk environmental data if the target JSON specification requires environmental metadata mapping alongside the block structures.

The Dynamic Bit-Packing Architecture: Litematica

The Litematica format (.litematic) introduces the concept of multi-region schematics and dynamic bit-packing, rendering it computationally the most complex format to decode. A single file contains a Regions compound tag, which acts as a dictionary of distinct spatial volumes, allowing multiple disconnected architectural elements to be stored within a single file. Each region possesses its own independent BlockStatePalette and BlockStates array.

Unlike the Sponge format, Litematica ensures that index 0 is strictly reserved for minecraft:air, regardless of whether air blocks are actively present within the geometric bounds. The BlockStates data is stored as a continuous long[] (an array of 64-bit integers). To compress the data to an absolute minimum, Litematica calculates the minimum number of bits required to represent the highest index in the palette.

If a region contains P unique block states, the bit-length b is determined by the formula: $b = \lceil \max(2, \lceil \log_2(P) \rceil) \rceil$. For example, if a region possesses 75 unique blocks, the bit-length b is set to 7, because $2^7 = 128$, which is sufficient to encode the index 75. The minimum bit-length is always artificially floored at 2 bits per entry to maintain structural integrity. These b -bit values are tightly packed into the 64-bit integers. Because b is rarely a clean divisor of 64, a single block's index will frequently span the boundary between two adjacent long integers. The extraction algorithm requires tracking the global bit-offset for each block, determining which long integers contain the overlapping data, and using extensive bitwise masking (&) and shifting (<<, >>) to reconstruct the original palette index.

Furthermore, Litematica regions specify their relative positions using negative and positive dimensions projected from a primary coordinate node. Resolving the final absolute coordinates for the JSON output requires calculating the enclosing boundary box and applying a relative

offset mapping to ensure all sub-regions align correctly in a unified Cartesian grid. Dedicated libraries such as `litemapy` abstract much of this deep bit-level complexity, allowing for direct iteration over regions and coordinate ranges without manually unwrapping the `long[]` boundaries.

The Layered Orthogonal Architecture: Bedrock `.mcstructure`

Bedrock Edition's native format introduces a layered approach to voxel storage to accommodate unique game mechanics, such as universal waterlogging. A single Cartesian coordinate can hold multiple blocks simultaneously—specifically, a primary structural block and a secondary liquid block.

The `.mcstructure` file utilizes a single global `block_palette` and stores spatial data inside a `block_indices` list. This list contains two separate arrays of integers, representing the primary and secondary layers. A value of `-1` within these arrays indicates an empty void space, instructing the game to ignore the coordinate during placement entirely.

The linear-to-spatial mapping formula for Bedrock structures is fundamentally different from the Java implementations. Given a size vector (`SZ`, `SY`, `SX`), the one-dimensional index `i` corresponds to the 3D coordinates via the formula:

$$i = (SZ \times SY \times X) + (SZ \times Y) + Z$$

Reversing this equation to extract explicit `X`, `Y`, `Z` coordinates requires sequential modulo and floor division operations against the dimension boundaries. The converter must evaluate both layers at every coordinate; if the secondary layer contains a valid fluid state, the target `jsonspec` architecture may need to combine the two palette entries into a unified waterlogged block state string, as the simplified JSON target schema does not inherently support stacked independent blocks at identical coordinate keys.

High-Performance Binary Architectures: Axiom and Create

Modern modding ecosystems have birthed highly specialized formats. The Axiom format (`.axiom`) utilizes a binary magic number `0x0AE5BB36` and relies on chunk-based volumetric storage rather than localized linear arrays. This allows the format to store massive architectural datasets with embedded thumbnail previews and extreme compression ratios.

Similarly, the Create mod employs a highly structured NBT format specialized in storing mechanical contraptions, prioritizing kinetic stress components and dynamic entity representations over static architectural blocks. A universal converter must detect these specific format signatures and route the data stream into dedicated decoders capable of parsing chunk-based or entity-driven binary blobs before attempting JSON conversion.

Entities, Block Entities, and State Normalization

Extracting the base block type is insufficient for modern voxel structures; the nuances of a luxury mansion or an intricate redstone contraption rely heavily on block states and nested tile entities.

The format of a `BlockState` identifier consists of a base namespace (e.g., `minecraft:oak_stairs`) appended with a set of comma-separated property key-value pairs surrounded by square brackets (e.g., `[facing=east,half=top]`). The universal converter must parse these strings and sort the property names alphabetically to ensure that identical blocks are not mapped to

multiple, redundant indices within the new JSON palette. For example, [half=top,facing=east] and [facing=east,half=top] must resolve to the exact same palette index to maintain maximum data deduplication.

BlockEntities (historically referred to as TileEntities) introduce another layer of complexity. These are NBT compounds containing arbitrary metadata for blocks that require additional state tracking, such as chest inventories, furnace smelting times, or command block execution strings. In the Sponge and Litematica formats, BlockEntities are stored as a separate NBT list of compounds mapped to explicit coordinates.

In the targeted jsonspec architecture, there is no explicit key or array designated for isolated block entities. Therefore, the converter must either discard this data—resulting in the catastrophic loss of chest contents and sign text—or dynamically append the NBT data directly to the string definition within the JSON palette. For example, a chest containing items could be registered in the new JSON palette as:

```
"minecraft:chest[facing=north]{Items:[{Slot:0b,id:\"minecraft:diamond\",Count:64b}]}"
```

While this methodology successfully preserves the required data within a flattened schema, it risks highly fragmenting the palette. If a schematic contains 1,000 chests, each with unique inventory configurations, the palette will expand by 1,000 unique entries, increasing the memory footprint of the final JSON object.

Target Schema Definition and Explicit Coordinate Mapping

The final output of the converter is a highly explicit, non-compressed JSON structure. The required layout mimics a raw architectural blueprint, completely abandoning the space-saving array index methodologies of the source binary formats. It contains a root object with three primary keys: name, palette, and blocks.

Key	Structural Type	Technical Description
name	String	The normalized string title of the schematic, extracted from the NBT metadata object.
palette	Object / Dictionary	A dictionary mapping string-based numerical indices (e.g., "0", "1") to fully qualified Minecraft resource locations and states (e.g., "minecraft:dark_oak_stairs[facing=south]").
blocks	Array of Objects	A collection of objects defining individual voxel placements. Each object contains integer values for x, y, and z, and a string block referencing a precise key in the palette.

This schema prioritizes strict interoperability and human readability over storage efficiency. Binary structures store coordinate data implicitly through the index of the block in a linear, continuous array. The jsonspec format, however, explicitly defines the X, Y, Z coordinates for

every single voxel placement.

For large-scale structural schematics, this explicit coordinate mapping causes an exponential, non-linear increase in file size. A moderately dense 100 \times 100 \times 100 schematic contains one million blocks. In a binary varint array (Sponge v2), this consumes roughly 1.5 to 2 megabytes of disk space. In the jsonspec format, one million explicit JSON objects, complete with string keys and formatting spaces, will consume upwards of 60 to 80 megabytes.

Therefore, the data mapping strategy must prioritize the aggressive culling of void and air spaces. Blocks mapping to `minecraft:air`, `minecraft:cave_air`, or `minecraft:void_air` must be excluded from the blocks array entirely to maintain manageable file sizes and prevent unnecessary parsing overhead during the loading phase.

Performance Profiling and Memory Management

Translating dense binary arrays into sparse JSON object arrays requires highly optimized linear traversal algorithms and extreme awareness of the host system's memory constraints.

Loading the entire final JSON structure into system RAM before writing to disk will inevitably trigger catastrophic out-of-memory (OOM) exceptions for larger structures. Standard JSON encoders, such as Python's native `json.dumps()`, attempt to allocate the entire string representation in memory simultaneously before initiating disk I/O.

To bypass this critical bottleneck, the converter must implement a chunked or streaming JSON serialization approach. The name and palette objects are written to the output file stream first. Subsequently, the blocks array is opened, and the intermediate representation yields one coordinate tuple at a time. The tuple is converted into a highly compact JSON dictionary string—such as `{"x": 12, "y": 5, "z": -4, "block": "3"}`—and appended directly to the open file stream, bypassing deep memory allocation.

Profiling this streaming pipeline is critical during development. Tools such as `memory_profiler` provide line-by-line consumption metrics, allowing developers to identify unexpected data retention during the varint decoding or bit-unpacking phases. For deeper, C-level extension profiling—especially relevant when interfacing with binary parsers or utilizing native `zlib` decompression calls—the `Memray` profiling tool is indispensable. `Memray` tracks allocations originating from native C extensions, revealing hidden bottlenecks that high-level Python profilers like `cProfile` often obscure.

The choice of unpacking library also vastly impacts performance. While the Python `struct` module provides direct C-level byte unpacking via `struct.unpack()`, it requires the developer to manually manage offset indices and chunk buffering. Higher-level libraries like `nbtlib` provide an elegant, pythonic dictionary interface, but their heavy reliance on Python object instantiation for every single NBT tag introduces significant memory overhead. In highly performant, production-grade environments, utilizing libraries written in Cython or Rust—such as `amulet-nbt` or `fastnbt`—allows for rapid, zero-copy serialization that drastically speeds up the time-to-first-token during the initial file ingestion phase.

Algorithm Design: The Universal Conversion Pipeline

To successfully build the universal schematic converter, a rigid, multi-stage data processing pipeline must be established. The pipeline is designed to abstract the variations in file types into an intermediate unified memory model, which is then streamed out to the explicit JSON format.

Stage 1: File Ingestion and Format Heuristics

The system receives an arbitrary binary file. It inspects the initial magic bytes to detect compression wrappers. If 1F 8B is detected, the stream is routed through a Gzip decompressor; if 78 is detected, a Zlib decompressor is engaged. Next, it utilizes heuristic analysis on the root NBT tags to identify the specific volumetric format:

- If Regions is present at the root, the format is identified as Litematica, and the dynamic bit-packing unpacker is queued.
- If Palette and BlockData are present, the format is identified as Sponge v2/v3, and the varint decoder is queued.
- If Blocks and Data byte arrays are present, the format is identified as Legacy MCEdit, and the DataFixerUpper legacy ID translation matrix is loaded.
- If block_indices is present, the format is identified as Bedrock .mcstructure, and the little-endian stream parser is engaged.

Stage 2: Abstraction and The Intermediate Representation (IR)

The data is translated into an Intermediate Representation (IR). The IR consists of a normalized string-based block palette and an asynchronous iterator that yields (X, Y, Z, StateString, BlockEntityNBT) tuples. For legacy formats, this stage applies the numerical ID to string mapping. For Bedrock formats, it evaluates the primary and secondary layers, merging them if necessary. Utilizing an iterator rather than pre-computing the entire 3D array in memory is the paramount architectural decision required to mitigate memory exhaustion.

Stage 3: Coordinate Resolution and Offset Calibration

As the iterator yields data, the universal converter calculates the explicit spatial coordinates based on the detected format's unique indexing formula. Once the local (X, Y, Z) coordinates are derived from the linear index, the global offset vector—defined in the schematic's metadata—is applied. If the schematic defines an origin offset of [-5, 12, -5], these values are algebraically added to the localized coordinates to ensure the final structure retains its original world-space anchoring.

Stage 4: Streaming JSON Serialization

The converter initiates the disk I/O process. The newly normalized, zero-indexed palette is serialized and written to the output file. The script then loops over the IR iterator. If the StateString resolves to a variant of air, the tuple is immediately discarded. Valid blocks are mapped to their new string-based palette index, formatted into the explicit {"x": X, "y": Y, "z": Z, "block": "ID"} dictionary structure, and appended directly to the open file stream.

Conclusion

The creation of an omni-directional schematic-to-JSON converter is a deeply sophisticated exercise in binary format normalization, bit-level memory unpacking, and high-performance data serialization. The extreme fragmentation of voxel blueprints requires a pipeline capable of processing uncompressed little-endian arrays, legacy numerical byte arrays, variable-length

integers, and dynamic bit-packed 64-bit boundaries. By deploying rigorous format heuristics, leveraging abstract iterators for memory stability, and streaming the output generation to prevent algorithmic bottlenecks, a robust software architecture can be achieved that seamlessly converts any esoteric voxel schematic into a clean, interoperable JSON specification.

Works cited

1. Minecraft NBT format - Just Solve the File Format Problem, http://justsolve.archiveteam.org/wiki/Minecraft_NBT_format
2. Named Binary Tag (NBT) | NeoForged docs, <https://docs.neoforged.net/docs/1.21.1/datastorage/nbt>
3. NBT format - Minecraft Wiki - Fandom, https://minecraft.fandom.com/wiki/NBT_format
4. NBT formaat - Minecraft Wiki - Fandom, https://minecraft.fandom.com/nl/wiki/NBT_formaat
5. Minecraft NBT (Named Binary Tag) format spec for Kaitai Struct, https://formats.kaitai.io/minecraft_nbt/
6. python - How to tell if a file is gzip compressed? - Stack Overflow, <https://stackoverflow.com/questions/3703276/how-to-tell-if-a-file-is-gzip-compressed>
7. How are zlib, gzip and zip related? What do they have in common and how are they different? [closed] - Stack Overflow, <https://stackoverflow.com/questions/20762094/how-are-zlib-gzip-and-zip-related-what-do-they-have-in-common-and-how-are-they>
8. GZip vs Deflate vs Zlib - Compression Formats and Headers Explained | DevToys Web Pro, <https://devtoys.pro/blog/gzip-deflate-zlib-formats>
9. zlib — Compression compatible with gzip — Python 3.14.7 documentation, <https://docs.python.org/3/library/zlib.html>
10. Bedrock mcstructure file format - GitHub Gist, <https://gist.github.com/tryashtar/87ad9654305e5df686acab05cc4b6205>
11. .mcstructure | Bedrock Wiki, <https://wiki.bedrock.dev/nbt/mcstructure>
12. About NBT (Named Binary Tag) - Bedrock Wiki, <https://wiki.bedrock.dev/nbt/nbt-in-depth>
13. vberlier/nbtlib: A python library to read and edit nbt data. - GitHub, <https://github.com/vberlier/nbtlib>
14. Loading and Editing Files — nbtlib 1.12.1 documentation - GitHub Pages, <https://vberlier.github.io/nbtlib/files/>
15. Chunk format - minecraft.fandom.com, https://minewiki.pl/Specjalna:Obcoj%C4%99zyczne/minecraft.fandom.com%2CChunk_format
16. Automatically generate Litematica file · Issue #111 - GitHub, <https://github.com/maruohon/litematica/issues/111>
17. Schematic file format for 1.13 and newer is not supported · Issue #40 · erich666/Mineways, <https://github.com/erich666/Mineways/issues/40>
18. WorldEdit/CHANGELOG.txt at version/7.4.x - GitHub, <https://github.com/EngineHub/WorldEdit/blob/version/7.4.x/CHANGELOG.txt>
19. Schematic-Specification/versions/schematic-2.md at master - GitHub, <https://github.com/SpongePowered/Schematic-Specification/blob/master/versions/schematic-2.md>
20. Util - Legacy Minecraft ID "Support" - Bukkit.org, <https://bukkit.org/threads/legacy-minecraft-id-support.487908/>
21. Resource location - Minecraft Wiki - Fandom, https://minecraft.fandom.com/wiki/Resource_location
22. GitHub - scrayosnet/shard-format: SHARD (Shard Highly Augmented Region Data) Format for Minecraft, <https://github.com/scrayosnet/shard-format>
23. SHARD Format – A New Take on Minecraft Region Data - ScrayosNET, <https://scrayos.net/justchunks-shard-format>
24. Which lesser known libraries saved your butt this year? : r/java - Reddit, https://www.reddit.com/r/java/comments/1pw69hq/which_lesser_known_libraries_saved_your_butt_this/
25. The Sponge Schematic Format Specification - GitHub, <https://github.com/SpongePowered/Schematic-Specification>
26. Schematic-Specification/versions/schematic-3.md at master · SpongePowered/Schematic-Specification - GitHub, <https://github.com/SpongePowered/Schematic-Specification>

<https://github.com/SpongePowered/Schematic-Specification/blob/master/versions/schematic-3.md> 27. Details on the litematica format? · Issue #53 - GitHub, <https://github.com/maruohon/litematica/issues/53> 28. litemapy - PyPI, <https://pypi.org/project/litemapy/> 29. Save space with bit arrays - Perl.com, <https://www.perl.com/article/save-space-with-bit-arrays/> 30. Bit Packing - CS 3410, <https://www.cs.cornell.edu/courses/cs3410/2024fa/notes/bitpack.html> 31. Getting started - Litemapy documentation - Read the Docs, <https://litemapy.readthedocs.io/en/latest/setup.html> 32. schem package - github.com/oriumgames/schem - Go Packages, <https://pkg.go.dev/github.com/oriumgames/schem> 33. AXIOM: Learning to Play Games in Minutes with Expanding Object-Centric Models, https://www.researchgate.net/publication/392314984_AXIOM_Learning_to_Play_Games_in_Minutes_with_Expanding_Object-Centric_Models 34. Schematicannon - Create Wiki - Fandom, <https://create.fandom.com/wiki/Schematicannon> 35. uberswe/mcnbt: MC-NBT is a Go library for working with Minecraft NBT (Named Binary Tag) data, specifically focused on schematic formats used in Minecraft. - GitHub, <https://github.com/uberswe/mcnbt> 36. Cross-format schematic converter (Bedrock/Java, WorldEdit, MCEdit, Litematica, Schematica, Amulet, Building Gadgets, Create) : r/feedthebeast - Reddit, https://www.reddit.com/r/feedthebeast/comments/1rdry66/crossformat_schematic_converter_bedrockjava/ 37. Forge Blockstate JSON - Minecraft Forge Documentation, <https://docs.minecraftforge.net/en/1.12.x/models/blockstates/forgeBlockstates/> 38. 1.8 rendering primer - GitHub Gist, <https://gist.github.com/williewillus/57d7093efa80163e96e0> 39. Connecting Blocks and Items to Models - Forge Documentation, <https://docs.minecraftforge.net/en/1.12.x/models/using/> 40. 1.8 - Minecraft Wiki - Fandom, <https://minecraft.fandom.com/hu/wiki/1.8> 41. MTS Editor - Page 2 - Luant Forums, <https://forum.luant.org/viewtopic.php?t=23724&start=25> 42. Handling Large JSON Files in Python: Efficient Read, Write, and Update Strategies, <https://dev.to/lovestaco/handling-large-json-files-in-python-efficient-read-write-and-update-strategies-3jgg> 43. Efficiently Processing Large JSON Files in Python Without Loading Everything Into Memory, <https://www.raydak.de/blog/2025-06-11-python-large-json/> 44. GitHub - karminski/streaming-json-py: A streamlined, user-friendly JSON streaming preprocessor, crafted in Python., <https://github.com/karminski/streaming-json-py> 45. How to manage large JSON files in Python efficiently and quickly - Sease, <https://sease.io/2021/10/how-to-manage-a-large-json-file-efficiently-and-quickly.html> 46. (12/22/20) Memory profiling python scripts with memory_profiler — Analytics and Integrative Machine Learning, <https://ncar.github.io/aiml-utils/memory.html> 47. Introduction to Memory Profiling in Python - DataCamp, <https://www.datacamp.com/tutorial/memory-profiling-python> 48. Python memory profiling and speed benchmarks for the Draft.js exporter - Thibaud's blog, <https://thib.me/python-memory-profiling-for-the-draft-js-exporter> 49. Memray is a memory profiler for Python - GitHub, <https://github.com/bloomberg/memray> 50. Python struct.pack and struct.unpack for Binary Data - DigitalOcean, <https://www.digitalocean.com/community/tutorials/python-struct-pack-unpack> 51. struct — Interpret bytes as packed binary data — Python 3.14.7 documentation, <https://docs.python.org/3/library/struct.html> 52. Python struct Pack and Unpack – Working with Binary Data - MangoHost, <https://mangohost.net/blog/python-struct-pack-and-unpack-working-with-binary-data/> 53. nbtlib - PyPI, <https://pypi.org/project/nbtlib/1.4.1a11/> 54. Edit Minecraft .dat File in Python - Stack Overflow, <https://stackoverflow.com/questions/73915085/edit-minecraft-dat-file-in-python> 55. Awesome-Minecraft/README.md at main - GitHub,

<https://github.com/Bytekron/Awesome-Minecraft/blob/main/README.md> 56.

uberswe/awesomeminecraft.com: A curated collection of 263+ resources spanning the entire Minecraft ecosystem. - GitHub, <https://github.com/uberswe/awesomeminecraft.com>